s🎓 **Algorithmic Problem Solving: Java Edition**

Topics: Hashing, Greedy Scheduling, Sliding Window logic.

🛠️ Problem 1: The Smart Campus Entry Tracker

The Real-Life Scenario

Greenwood University has installed an IoT-based infrared scanner at the entrance of the "Great Hall" for a career fair. Every time a student passes through the gate, their RFID tag is scanned.

The system logs two types of events:

- **Entry (+ ID):** The student enters the hall.
- **Exit (- ID):** The student leaves the hall.

The Issue: The Security Department has noticed that some students are "tailgating" (slipping through the door behind someone else). This causes a logic error where the system records a student **leaving** the hall who was never recorded **entering**.

Input Format

- First line: N (Number of logs)
- Next N lines: + ID or - ID

Output Format

- Final count of students inside or INVALID.

Java Solution

Java

```
import java.util.*;

public class EventTracker {

   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      int n = sc.nextInt();

      HashSet<Integer> inside = new HashSet<>();

      boolean error = false;


      for (int i = 0; i < n; i++) {

         String op = sc.next();

         int id = sc.nextInt();
```

```
        if (op.equals("+")) {
            inside.add(id);
        } else {
            if (!inside.contains(id)) {
                error = true;
            }
            inside.remove(id);
        }
    }
    System.out.println(error ? "INVALID" : inside.size());
  }
}
```

📅 Problem 2: The Freelance Project Optimizer

The Real-Life Scenario

You are a freelance software developer with a list of N pending contracts. Each contract has a specific **Time Requirement** (days to complete) and a **Strict Deadline** (the day it must be submitted).

You work sequentially; you can only work on **one** project at a time. You cannot submit a project even one minute past its deadline. Your goal is to maximize your portfolio by completing the highest number of tasks possible.

Input Format

- First line: N
- Next N lines: duration deadline

Output Format

- Maximum tasks completed.

Java Solution

Java

```
import java.util.*;


class Project {
  int time, limit;
```

```
    Project(int t, int l) { this.time = t; this.limit = l; }
}

public class PortfolioManager {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        Project[] list = new Project[n];

        for(int i=0; i<n; i++) list[i] = new Project(sc.nextInt(), sc.nextInt());

        // Greedy Strategy: Sort by Deadline
        Arrays.sort(list, (a, b) -> a.limit - b.limit);

        int timePassed = 0, count = 0;
        for(Project p : list) {
            if(timePassed + p.time <= p.limit) {
                timePassed += p.time;
                count++;
            }
        }
        System.out.println(count);
    }
}
```

🛡️ Problem 3: The "Flash-Chat" Anti-Spam Filter

The Real-Life Scenario

A messaging app, "Flash-Chat," prevents bot attacks by enforcing a **Cooldown Rule (K)**. If a message is sent at time T, the system blocks any further messages from that user until at least K seconds have passed.

Critical Rule: Only **successfully sent** messages reset the cooldown timer. If a message is blocked, the "last successful time" remains the same.

Input Format

- First line: N (Total messages) and K (Cooldown)
- Second line: N space-separated timestamps.

Output Format

- Total number of blocked messages.

Java Solution

Java

```
import java.util.*;


public class SpamFilter {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      if(!sc.hasNextInt()) return;


      int n = sc.nextInt();
      int k = sc.nextInt();
      int[] logs = new int[n];
      for(int i=0; i<n; i++) logs[i] = sc.nextInt();


      if(n == 0) { System.out.println(0); return; }
      int blocked = 0;
      int lastSuccess = logs[0]; // First message always passes
      for(int i = 1; i < n; i++) {
         if(logs[i] - lastSuccess < k) {
            blocked++;
         } else {
            lastSuccess = logs[i]; // Reset timer on success
         }
          System.out.println(blocked);
}
 }
```